DEEP SYSTEM DESIGN DOCUMENT

AI Job Search Agent

Every component explained — for the complete beginner and the engineer

🧠

7 Core Components

Each explained in plain English AND full technical depth

🔄

Complete Data Flow

Exactly what data goes in and comes out of every component

⚖️

Every Design Decision

Why each technology was picked over every alternative

REPO	STACK	COST	DATE
github.com/sushant17112000/ai-job-agent	Python · Playwright · Groq · GitHub Actions	$0 / month	March 2026

Table of Contents

Each component has a layman section, a technical section, input/output, tech choices, and failure modes.

FOUNDATIONS

0 What is System Design? (The Big Picture) — 3
How all 7 components connect, and what "system design" means

THE 7 COMPONENTS

1 GitHub Actions — The Alarm Clock — 4
.github/workflows/daily_job_search.yml

2 CV Parser — The Resume Reader — 6
src/cv_parser.py · pdfplumber · Groq API

3 Playwright Scrapers — The Browser Robot — 8
src/scrapers/ · Chromium · anti-detection

4 Groq LLM API — The AI Brain — 11
Llama 3.3 70B · prompt engineering · rate limits

5 Job Matcher — The Judge — 13
src/job_matcher.py · scoring · experience filter · dedup

6 Excel Generator — The Report Writer — 16
src/excel_generator.py · pandas · openpyxl

7 GitHub Uploader — The Delivery Person — 18
src/github_uploader.py · git CLI · CI authentication

SYSTEM-WIDE DESIGN

8 Full End-to-End Data Flow — 19
What every byte of data looks like at each stage

9 Error Handling Philosophy — 21
How the system fails gracefully so one bad thing never kills everything

10 Complete Technology Decision Log — 22
Every choice, every alternative, every reason

0

FOUNDATION

What Is System Design? The Big Picture.

Before we dive into components, let's understand what "system design" actually means and see how all seven pieces of this agent fit together like a factory assembly line.

Think of it like a factory that makes a product every day.
Raw material goes in one end (your CV + job sites). Each station transforms it. A finished product comes out the other end (your Excel report). System design is the blueprint for that factory — what each station does, how they talk to each other, and what happens when a machine breaks.

The Big Picture — All 7 Components

This is the factory floor. Each box is one component. Data flows left to right, top to bottom.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          GITHUB ACTIONS RUNNER                           │
│                    (A free cloud computer rented from GitHub)            │
│                    Wakes up at 7 PM IST every day automatically          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  YOUR CV (PDF)                                                           │
│      │                                                                   │
│      ▼                                                                   │
│  ┌──────────────────────┐                                                │
│  │  COMPONENT 2         │  ─── sends CV text ──►  GROQ AI (Component 4) │
│  │  CV PARSER           │  ◄── gets profile JSON ──                     │
│  │  "The Reader"        │                                               │
│  └──────────────────────┘                                                │
│          │                                                               │
│          │  profile: {name, skills, experience_years, target_roles, ...} │
│          ▼                                                               │
│  ┌──────────────────────────────────────────────────┐                   │
│  │  COMPONENT 3 — SCRAPERS  "The Browser Robots"    │                   │
│  │  (all 4 run at the same time, simultaneously)    │                   │
│  │                                                  │                   │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌────┐  │                 │
│  │  │ LinkedIn │  │  Naukri  │  │ IIMJobs  │  │ JS │  │                 │
│  │  │ ~128 jobs│  │  0 jobs  │  │  0 jobs  │  │ 0  │  │                 │
│  │  └──────────┘  └──────────┘  └──────────┘  └────┘  │                 │
│  └──────────────────────────────────────────────────┘                   │
│          │                                                               │
│          │  list of raw job dicts: [{title, company, url, ...}, ...]     │
│          ▼                                                               │
│  ┌──────────────────────┐                                                │
│  │  COMPONENT 5         │  ─── sends batches of 10 ──►  GROQ AI         │
│  │  JOB MATCHER         │  ◄── gets scores + reasons ──                 │
│  │  "The Judge"         │                                               │
│  └──────────────────────┘                                                │
│          │                                                               │
│          │  ranked list of matched jobs with scores                      │
│          ▼                                                               │
│  ┌──────────────────────┐                                                │
│  │  COMPONENT 6         │                                               │
│  │  EXCEL GENERATOR     │──► reports/2026-03-03_jobs.xlsx               │
│  │  "The Report Writer" │                                               │
│  └──────────────────────┘                                                │
│          │                                                               │
│          ▼                                                               │
│  ┌──────────────────────┐                                                │
│  │  COMPONENT 7         │──► git push → github.com/sushant17112000/     │
│  │  GITHUB UPLOADER     │              ai-job-agent/reports/            │
│  │  "The Delivery Guy"  │                                               │
│  └──────────────────────┘                                                │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

L WHY IS THIS SPLIT INTO 7 COMPONENTS? WHY NOT ONE BIG PROGRAM?

Imagine building a car. You wouldn't want one giant machine that tries to do everything — weld the body, install the engine, AND paint it. You'd have separate stations. If the painting station breaks, the welding station keeps running. Same logic here.

Each component does one job and hands off to the next. If the LinkedIn scraper gets blocked by LinkedIn today, only that one component returns nothing — the rest of the pipeline still runs with whatever data it has.

T SOFTWARE ARCHITECTURE PATTERN: PIPELINE WITH FAULT ISOLATION

This follows a **pipeline pattern** — each stage transforms data and passes it to the next. Combined with **fault isolation**: every component catches its own exceptions and returns a safe default (usually an empty list) rather than letting an exception propagate upward and kill the run.

The orchestrator (`main.py`) only aborts on truly fatal conditions (no API key, CV file missing). Everything else is a warning, not a crash.

1

COMPONENT 1

GitHub Actions

The Alarm Clock

This is the component that starts everything. It's the reason you don't have to do anything — it wakes up at exactly 7 PM IST every single day and triggers the entire pipeline automatically.

Think of it as a very reliable alarm clock that lives in the cloud.
When it rings, instead of waking you up, it wakes up a temporary computer, runs your entire agent start to finish, saves the result, then shuts the computer back down — all without you lifting a finger.

GitHub Actions — The Alarm Clock

`.github/workflows/daily_job_search.yml`

L PLAIN ENGLISH EXPLANATION

GitHub is a website where programmers store their code. It also offers a free feature called "GitHub Actions" — essentially, you can tell GitHub: *"At 7 PM every day, spin up a computer, run my program, and shut it down."*

That's exactly what happens here. There's a file in your project called `daily_job_search.yml` — a recipe/instruction list written in YAML format (a simple config language). This file tells GitHub:

- When to run (7 PM IST every day, OR whenever you click "Run Workflow" manually)
- What kind of computer to use (Ubuntu Linux, the most common server OS)
- What to install (Python, a browser, all the libraries)
- What to execute (your Python program)
- What to do with the output (save the Excel file and push it to the repo)

GitHub provides this for free on public repos and gives 2,000 minutes/month free on private repos. Your agent uses roughly 4 minutes per run × 30 days = ~120 minutes/month — well within free limits.

T TECHNICAL DEEP DIVE

Trigger mechanism: The `schedule` trigger uses POSIX cron syntax: `30 13 * * *` — field by field: minute=30, hour=13, day=any, month=any, weekday=any. 13:30 UTC = 19:00 IST (UTC+5:30). The `workflow_dispatch` trigger adds a manual "Run workflow" button in the GitHub UI.

Runner environment: `ubuntu-latest` is a GitHub-hosted runner — a fresh ephemeral Ubuntu VM spun up exclusively for your job. It has Python 3.x, git, curl pre-installed. It's destroyed after the job completes. Nothing persists between runs except what's committed back to the repo.

GITHUB_TOKEN: Every run automatically gets a short-lived `GITHUB_TOKEN` scoped to your repo. By setting `permissions: contents: write` in the workflow, the token gains push access — used by the final git push step to commit the Excel file back.

Secret injection: `GROQ_API_KEY` is stored encrypted in GitHub Secrets (Settings → Secrets → Actions). It's injected as an environment variable at runtime — never visible in logs, never stored in code.

📥 TRIGGER INPUT

- Cron schedule (13:30 UTC daily)
- OR manual workflow_dispatch click
- GROQ_API_KEY from GitHub Secrets
- Current repo contents (cv/, src/, main.py)

📤 WHAT IT PROVIDES

- A clean Ubuntu VM with Python 3.11
- Playwright + Chromium installed
- All Python libraries installed
- GROQ_API_KEY available as env var

- git configured with write access

⚖️ WHY GITHUB ACTIONS? WHY NOT SOMETHING ELSE?

Option	Cost	Setup Complexity	Why Rejected
GitHub Actions ✓	Free	None — YAML file in repo	— (chosen)
AWS Lambda + EventBridge	Free tier then $	High — IAM, VPC, layers	No persistent storage; Playwright too large for Lambda (250MB limit)
GCP Cloud Run + Cloud Scheduler	Free tier then $	High — service accounts, billing	Requires credit card; overkill for a single daily job
Cron on a VPS (DigitalOcean etc.)	$4–6/month	Medium — SSH, systemd	Costs money; server maintenance overhead
Your own laptop	Free	None	Laptop must be on at exactly 7 PM every day — unreliable
PythonAnywhere scheduler	Free (limited)	Low	Cannot install Playwright; blocked from running headless browsers

⚠️ FAILURE MODES

WHAT CAN FAIL	SEVERITY	HOW IT'S HANDLED
Cron schedule delayed by GitHub queue	Low	Can be 0–15 min late during GitHub peak hours; acceptable
Workflow file has YAML syntax error	Fatal	Workflow doesn't start at all; fix YAML and push
GROQ_API_KEY secret not set	Fatal	main.py checks for it and exits with clear error message
pip install fails (bad package version)	Fatal	Workflow fails at step 3; pinned versions in requirements.txt prevent this
git push rejected (remote ahead)	Medium	Workflow uses git pull --rebase before pushing to handle bot commits

2

COMPONENT 2

CV Parser

The Resume Reader

This component reads your PDF resume and turns it into a structured data format that the AI can reason about. It's the first thing that runs and sets the context for everything else.

Think of it as a very smart assistant who reads your entire CV in 2 seconds and fills out a form with all the important details.
You hand them a 2-page PDF. They hand back a filled form: name, what kind of job you want, your skills, how many years of experience you have, your education, and where you want to work.

CV Parser — The Resume Reader

src/cv_parser.py

Stage 1: PDF → Plain Text (pdfplumber)

L **LAYMAN: READING THE PDF**

A PDF file isn't just text — it's a complex format that stores text, fonts, images, and layouts in a structured way. You can't just open a PDF and read the words like a plain text file. You need a special tool to extract the text.

This is what **pdfplumber** does. It opens your PDF, goes through every single page, and extracts all the written text, then stitches it all together. The result is one long string of all the words in your CV — just like copying and pasting your CV into Notepad.

T **TECHNICAL: PDFPLUMBER PDF EXTRACTION**

`pdfplumber` uses `pdfminer.six` under the hood — a low-level PDF parser that models the PDF coordinate space and character positions. It re-constructs reading order by analyzing x/y coordinates of each character, making it far more accurate for multi-column CVs than naive PDF readers.

`page.extract_text()` returns a string per page, concatenated with double newlines between pages. If every page returns `None` (e.g., a scanned-image-only PDF where text isn't embedded), a `CVParseError` is raised immediately with a user-readable message.

```
with pdfplumber.open(pdf_path) as pdf:
    for page in pdf.pages:
        text = page.extract_text()
        if text:
            pages_text.append(text)
full_text = "\n\n".join(pages_text)
```

Stage 2: Plain Text → Structured Profile (Groq AI)

L **LAYMAN: THE AI READS YOUR CV**

Now we have the raw text of your CV. But it's just a wall of words. We need to understand *what it means* — what roles you're targeting, how many years of experience you have, what your skills are.

This is where AI comes in. We send your entire CV text to Groq's AI model (Llama 3.3 — a very capable free AI) and ask it to read it and fill out a form. The "form" is what programmers call JSON — a structured format that computers can read easily.

The AI understands context like a human does. It knows "5 years at TCS as Senior Analyst" means experience_years = 5. It knows "Python, SQL, Tableau" are skills. It doesn't need the CV to say "SKILLS:" in big letters above a list — it figures it out from context.

T TECHNICAL: PROMPT DESIGN & DEFENSIVE PARSING

Two-message prompt structure:

- System message: "*Return ONLY valid JSON with no explanation and no markdown code fences.*" — constrains the model's output format strictly.
- User message: Full CV text with an explicit schema listing the exact keys and types expected.

Temperature = 0.1: Near-deterministic output. Higher temperatures introduce randomness which could produce inconsistent JSON keys across retries.

Three defensive layers after receiving the response:

1. `_strip_markdown_fences()` — some models add ` ```json ``` ` despite instructions. We split on ` ``` ` and extract the content between fences.
2. `json.loads(raw)` — strict parse. Any deviation from valid JSON throws `JSONDecodeError`, triggering a retry.
3. `profile.setdefault(key, default)` for every required field — guarantees downstream code never hits a `KeyError` even if the model omits a field.

Retry logic: 2 attempts, exponential backoff (sleep 2s before attempt 2). After both fail, raises `CVParseError` — the only truly fatal error that stops the pipeline.

📥 INPUT

- Path to `cv/resume.pdf`
- Initialized Groq client object

📤 OUTPUT — CV_PROFILE DICT

- `name` : "Sushant Bhat"
- `target_roles` : ["Product Manager", ...]
- `skills` : ["Python", "SQL", ...]
- `experience_years` : 3
- `education` : "B.Tech, NIT"
- `preferred_locations` : ["Bengaluru", ...]

⚖️ TECHNOLOGY CHOICES

Choice	What we use	Alternatives	Why
PDF extraction	**pdfplumber**	PyPDF2, pypdf, pymupdf, textract	Best multi-column layout handling. pdfplumber preserves reading order for complex CV layouts. PyPDF2 scrambles column order.
CV understanding	**Groq AI (LLM)**	Regex, spaCy NER, NLTK	CVs have no standard format. Regex and NLP rules break on any deviation. LLM understands context like a human.

Output format	**JSON**	XML, CSV, Python dataclass	JSON is natively supported by Python's `json` module, serializable, human-readable, and what LLMs naturally produce.

⚠️ FAILURE MODES

WHAT CAN FAIL	SEVERITY	HOW IT'S HANDLED
PDF is a scanned image (no text layer)	**Fatal**	pdfplumber returns empty string → CVParseError raised → sys.exit(1)
Groq returns JSON with a markdown fence	**None**	_strip_markdown_fences() removes it before json.loads()
Groq returns invalid JSON	**Low**	Retry up to 2 times with 2s backoff; raise CVParseError if all fail
Groq omits a required field (e.g. "skills")	**None**	profile.setdefault("skills", []) fills in empty list; no crash
target_roles returned as a string not list	**None**	Explicit isinstance check wraps it in a list: ["Product Manager"]

3

COMPONENT 3

Playwright Scrapers

The Browser Robots

These are four automated "robots" that each open a real browser, visit a job portal, scroll through listings, and collect job data — exactly like a human would, but 100× faster and without getting bored.

Imagine hiring 4 interns and telling each one to go to a different job website, search for your target roles, and copy every new job listing onto a notepad. They all go at the same time (simultaneously), so you get all 4 portals searched in the time it would take to search one. That's this component.

Playwright Scrapers — The Browser Robots

src/scrapers/ — base_scraper.py + 4 portal-specific files

What is Web Scraping?

L LAYMAN: WHAT IS WEB SCRAPING?

Every website is made of HTML — a markup language that describes what's on the page. When your browser opens LinkedIn, it downloads that HTML and renders it into the visual page you see.

Web scraping means: instead of a human reading what the browser renders, a program reads the HTML and extracts specific pieces of information — job titles, company names, locations, URLs.

The problem: Modern websites like LinkedIn load their job listings using JavaScript *after* the initial HTML loads. If you just download the HTML directly (using Python's `requests` library), you get an empty shell with no job listings. You need a real browser that can execute the JavaScript and wait for the listings to appear.

That's exactly what **Playwright** does — it is a real Chrome browser that a program can control. It opens LinkedIn, waits for the jobs to load, then reads the HTML and extracts the data.

T TECHNICAL: WHY PLAYWRIGHT

LinkedIn, Naukri, IIMJobs, and Jobstreet are all Single-Page Applications (SPAs) built with React or similar frameworks. The initial HTTP response contains essentially just a root `<div id="app">`. Job listing HTML is injected by JavaScript after the React component tree mounts and completes API calls to backend services.

Playwright launches a real Chromium process, attaches to a DevTools Protocol (CDP) session, and provides an async Python API to control it. It genuinely executes JavaScript, fires `XHR`/`fetch` requests, handles dynamic imports, and waits for the DOM to settle — exactly matching what a human browser does.

Alternatives like `requests` + BeautifulSoup, `httpx`, or even `Scrapy` cannot execute JavaScript and therefore cannot access SPA-rendered content without reverse-engineering private APIs.

Anti-Detection — The Stealth Layer

L LAYMAN: WHY DO JOB SITES TRY TO BLOCK ROBOTS?

Job sites spend money on their servers. If thousands of bots hit them every day downloading all their job listings, it slows the site down and costs them more money. So they try to detect and block automated browsers.

They look for telltale signs that the visitor isn't a real human: a specific flag that Chrome sets when it's being controlled by a program, a fake user agent string, or a viewport size that no real person would have.

Our scraper pretends to be a real human by: using a real browser user agent string (the identity string Chrome sends to websites), using a realistic screen size, and hiding the "I am being controlled by a robot" flag that Playwright sets by default.

T **TECHNICAL: STEALTH CONTEXT IMPLEMENTATION**

Three specific anti-detection measures are applied to every browser context:

1. navigator.webdriver override:
By default, when Playwright controls Chrome, `navigator.webdriver` is set to `true`. Anti-bot scripts check this property immediately on page load. We override it using `page.add_init_script()` — a script injected before any page JavaScript runs:

```
await context.add_init_script("""
    Object.defineProperty(navigator, 'webdriver', {
        get: () => undefined
    });
""")
```

2. Randomized User-Agent:
We maintain a list of 5 real desktop browser UA strings (Chrome on Windows, macOS, etc.). One is picked randomly per browser context. This prevents fingerprinting by UA pattern.

3. Randomized Viewport:
A random viewport between 1200–1920px wide and 768–1080px tall is assigned. Fixed viewports are a common bot signal.

4. Linux-specific Chrome flags (required for GitHub Actions):

```
args=[
    "--no-sandbox",              # CI/Docker environments lack kernel sandbox
    "--disable-setuid-sandbox",  # complement to above
    "--disable-dev-shm-usage",   # /dev/shm is 64MB in GitHub Actions; Chrome needs more
    "--disable-gpu",             # no GPU in headless runners
]
```

The Four Scrapers

L **LAYMAN: WHAT DOES EACH SCRAPER DO?**

Each scraper knows the specific layout of its own website. LinkedIn's HTML is different from Naukri's. Each scraper is taught the exact pattern to look for on its site — like telling each intern exactly which column on which table to read.

Scraper	URL Strategy	Key Technical Detail	Status

LinkedIn	`f_TPR=r86400` param — last 24h only	Returns [] if <3 job cards detected (block indicator). Selectors: `ul.jobs-search__results-list li`	Working ~128/run
Naukri	Slug URL: `/product-manager-jobs-in-bengaluru`	`networkidle` wait + 2s extra + Escape key to dismiss modal	Blocked (0 jobs)
IIMJobs	India-wide URL; city post-filter in location text	No city in URL slug; filter after scraping by checking if location string contains target city	Blocked (0 jobs)
Jobstreet	Singapore-only: `/jobs/{slug}-jobs`	Relative URLs (e.g. `/job/123`) prepended with `https://www.jobstreet.com.sg`	Blocked (0 jobs)

Concurrency — All 4 Run Simultaneously

L **LAYMAN: RUNNING ALL 4 AT THE SAME TIME**

Instead of checking LinkedIn, waiting for it to finish, then checking Naukri, waiting, then IIMJobs... all four scrapers are launched at exactly the same moment and run in parallel. This means the total scraping time is equal to whichever portal takes the longest — not the sum of all four.

If LinkedIn takes 2 minutes and the others take 30 seconds each, the whole thing finishes in 2 minutes — not 3.5 minutes.

T **TECHNICAL: ASYNCIO.GATHER() CONCURRENCY**

Playwright's Python API is fully asynchronous (uses Python's `asyncio` event loop). All four scraper `scrape()` methods are coroutines (defined with `async def`). `asyncio.gather()` schedules all four concurrently on the same event loop:

```
results = await asyncio.gather(
    safe_scrape(LinkedInScraper(),   job_titles, CITIES),
    safe_scrape(NaukriScraper(),     job_titles, CITIES),
    safe_scrape(IIMJobsScraper(),    job_titles, CITIES),
    safe_scrape(JobstreetScraper(),  job_titles, CITIES),
)
```

This is not true parallelism (multiple CPU cores) — it's cooperative multitasking. While one scraper is awaiting a network response (I/O-bound), the event loop runs another scraper. For I/O-bound tasks like web scraping, this is equivalent in practice to true parallel execution.

Each scraper is wrapped in `safe_scrape()` which catches all exceptions and returns `[]` — preventing a single scraper failure from cancelling others in the gather group.

Why are 3 of 4 Scrapers Returning 0 Jobs?

L **LAYMAN: THE IP ADDRESS PROBLEM**

Every device connected to the internet has an IP address — a unique number that identifies it, like a postal address for internet traffic. When a website gets a request, it can see the IP address it came from.

GitHub's servers (which run your agent) all have IP addresses from Amazon's cloud (AWS). Job sites like Naukri know these IP ranges belong to data centers — not real human users at home. So they block those IPs entirely, returning a blank page or an error.

It's like a nightclub that checks your area code before letting you in. If you're calling from a business district instead of a residential area, they assume you're a bot and don't let you in.

The fix is to use official job search APIs (like Adzuna) that are designed to be accessed by programs — they give you the same data through a proper door, not through the scraping window.

T TECHNICAL: CGNAT / DATACENTER IP FINGERPRINTING

GitHub Actions runners use IP addresses from known AWS/Azure CIDR ranges (e.g. `13.x.x.x`, `40.x.x.x`). Indian job portals (Naukri, IIMJobs) and Jobstreet maintain blocklists of datacenter IP ranges — these portals receive enormous amounts of scraping traffic and have sophisticated bot-detection.

The block happens at the HTTP layer (403 response) or by serving a blank/CAPTCHA page. Our scrapers detect 0 job cards and return `[]` gracefully. The fix is API-based access (Adzuna, JSearch) which routes through approved API gateways.

📥 INPUT (EACH SCRAPER)

- `job_titles` list from cv_profile
- `CITIES` dict from config.py
- Playwright browser instance

📤 OUTPUT (EACH SCRAPER)

- List of job dicts, each containing:
- `title, company, location`
- `url` (absolute)
- `description_snippet`
- `date_posted, portal`

4

COMPONENT 4

Groq LLM API

The AI Brain

This is the intelligence layer — the AI that reads your CV and scores jobs. It's not a component you write; it's a service you call. Understanding how it works and why we chose it is critical.

Think of it as a brilliant consultant you can call on the phone at any time, for free.
You read them your CV, they memorize it. Then you read them a job description, and they immediately tell you: "That's an 85/100 match because your PM skills align perfectly, but the location isn't ideal." They're fast, always available, and don't charge you.

Groq LLM API — The AI Brain

Used by: `cv_parser.py` and `job_matcher.py`

What is an LLM? What is Groq?

L LAYMAN: WHAT IS A LARGE LANGUAGE MODEL?

An LLM (Large Language Model) is a type of AI that's been trained on enormous amounts of text — books, websites, research papers, code. It learned patterns in how language works so well that it can now understand questions and generate helpful, human-like responses.

ChatGPT is an LLM. Claude is an LLM. The one we use is called **Llama 3.3 70B** — built by Meta (Facebook's parent company), open-source, and extremely capable.

Groq is a company that built special computer chips (called LPUs — Language Processing Units) designed specifically to run LLMs incredibly fast. They offer free access to these models as a service. We call their API, send them text, they run it through Llama 3.3 and send back a response — all in about 1-2 seconds.

Why is it free? Groq is trying to build market share. Offering a generous free tier (14,400 requests per day) attracts developers. We use maybe 20 of those 14,400 per day — it's a rounding error to them.

T TECHNICAL: LLM API ARCHITECTURE

Model: `llama-3.3-70b-versatile` — a 70 billion parameter instruction-tuned model. "70B parameters" means the model has 70 billion numerical weights that collectively encode its understanding of language. The "versatile" variant is specifically fine-tuned for instruction following — critical for our structured JSON output requirement.

API Interface: Groq's Python SDK wraps a REST API compatible with OpenAI's Chat Completions format. Every call sends an array of messages with `role` (system/user/assistant) and `content` fields.

```
response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {"role": "system", "content": "Return ONLY valid JSON..."},
        {"role": "user",   "content": "CV text or job batch here..."},
    ],
    temperature=0.1,   # near-deterministic
    max_tokens=2048,   # sufficient for scoring 10 jobs
)
```

Temperature: A value 0–2 controlling randomness. At 0.0, the model always picks the highest-probability token (fully deterministic). At 2.0, it's highly random. We use 0.1 — almost deterministic but with tiny variance to avoid pathological repetition loops.

Free tier limits: 14,400 requests/day, ~30 requests/minute. Our pipeline makes ~15–20 calls per run. We add 1-second sleeps between scoring batches to stay well under the RPM limit.

Why Groq Specifically? The Provider Journey

⚖️ LLM PROVIDER EVALUATION — FULL HISTORY

Provider	Model Tried	Result	Root Cause
Anthropic (Claude API)	claude-sonnet-4-6	Rejected by user	User has Claude Pro (chat subscription) which does NOT include API access. API billing is separate. $15/1M tokens.
Google (google-generativeai SDK)	gemini-1.5-flash	404 error	SDK is deprecated. gemini-1.5-flash doesn't exist on this account — only 2.x models available.
Google (google-genai SDK)	gemini-2.0-flash	429 quota=0	Google AI Studio free tier for text generation has 0 RPD quota in India. The free tier only works in specific regions. Confirmed by API diagnostic script.
Groq	**llama-3.3-70b-versatile**	Working perfectly	Genuinely free, no regional restrictions, no credit card, 14,400 req/day, works from GitHub Actions IPs.

📥 INPUT (EACH API CALL)

- System message (format instruction)
- User message (CV text OR job batch)
- temperature: 0.1
- max_tokens: 1024 (CV) or 2048 (jobs)

📤 OUTPUT

- JSON string (profile OR scores array)
- Sometimes wrapped in markdown fences
- Sometimes fields are strings not ints
- Defensive parsing handles all edge cases

⚠️ LLM FAILURE MODES (AND WHY THEY'RE COMMON)

WHAT FAILS	FREQUENCY	HOW IT'S HANDLED
Returns markdown-fenced JSON (```json...```)	~10% of calls	_strip_markdown_fences() removes it before parsing
match_score returned as "85" (string) not 85 (int)	~5% of calls	_safe_score() coerces via int(float(str(...)))
Field values swapped (reason in score field)	~1% of calls	_safe_score() fails to coerce string → returns 0 → job filtered out
Rate limit exceeded (429)	Rare	Retry with exponential backoff; 1s sleep between batches prevents hitting RPM limit

Returns non-array JSON (e.g. single object)	**Rare**	isinstance check raises ValueError → retry

5

COMPONENT 5

Job Matcher

The Judge

This is the brain of the operation. It takes all the raw jobs collected by the scrapers, cleans them up, sends them to the AI for scoring, applies hard filters, and produces the final ranked shortlist.

Think of it as a very strict and fair judge reviewing 128 CVs against your profile.
First, the judge removes duplicates (same job posted twice). Then, using your profile, they give each job a score and a written reason. Any job requiring more experience than you have gets an automatic zero — no matter how good the title sounds. The rest are ranked best-first.

Job Matcher — The Judge

src/job_matcher.py

Stage 1: Deduplication

L LAYMAN: REMOVING DUPLICATES

Sometimes the same job appears on a portal multiple times with slightly different web addresses. For example, Naukri adds tracking codes to their URLs: `?src=search&applyRedirect=true`. The same job might appear as:

- `naukri.com/job/pm-role-123?src=search`
- `naukri.com/job/pm-role-123?src=recommended`

These are the same job! If we didn't deduplicate, we'd send the AI the same job twice, waste API calls, and it might appear twice in your report. So we strip off the tracking part of the URL and compare just the core address.

T TECHNICAL: URL NORMALIZATION

Python's `urllib.parse` module provides `urlparse()` and `urlunparse()`. We parse each URL into its components (scheme, netloc, path, params, query, fragment), strip `query` and `fragment` to zero, and reconstruct the URL. This is the canonical key for deduplication via a `set()`:

```
def _normalize_url(url: str) -> str:
    parsed = urlparse(url)
    return urlunparse(parsed._replace(query="", fragment=""))

seen = set()
unique = []
for job in jobs:
    key = _normalize_url(job.get("url", ""))
    if key and key not in seen:
        seen.add(key)
        unique.append(job)
```

Set lookup is O(1) — checking a set of 128 URLs is essentially instant regardless of size.

Stage 2: Batched AI Scoring

L LAYMAN: SENDING JOBS TO THE AI IN GROUPS OF 10

Imagine you had to ask a consultant to rate 128 jobs against your profile. Instead of calling them 128 times (one job per call), you call them 13 times and read 10 jobs at a time. They rate all 10 in one go and give you 10 scores back. Much more efficient.

That's exactly what we do. We group 128 jobs into batches of 10 (so 13 groups), call the AI once per group, and get back a list of 10 scores. Between each call we wait 1 second so we don't overwhelm the AI service.

Each score comes with a short written reason — "This role perfectly matches your 3 years of product management experience and the location is in Bengaluru, which is your first preference."

T TECHNICAL: BATCH ARCHITECTURE & PROMPT ENGINEERING

Why 10 per batch? Balancing two constraints: (1) minimizing number of API calls (fewer = lower latency) and (2) keeping each prompt within the model's effective attention span. At 10 jobs, each prompt is ~800–1200 tokens — well within the 32K context window, but large enough that the model can maintain consistent scoring criteria across all 10 within one generation.

The scoring prompt structure:

```
CANDIDATE PROFILE:          ← gives the AI the evaluation lens
- Experience: 3 years
- Target Roles: [Product Manager, ...]
- Skills: [Python, SQL, ...]

HARD RULES (override everything):     ← injected with real values
- Candidate has 3 years experience
- Any job requiring >5 years → score 0
- Senior/Director/VP/CTO → score 0

SCORING RUBRIC:             ← tells the AI what numbers mean
- 80-100: Excellent match
- 60-79:  Good match
- 40-59:  Partial match
- 0-39:   Poor match

JOBS TO SCORE:              ← the actual 10 jobs
Job 0: Title / Company / Location / Snippet
Job 1: ...
...

Return JSON array:          ← explicit output format
[{"job_index": 0, "match_score": 85, "match_reason": "..."}, ...]
```

Why sequential, not concurrent? Groq's free tier allows ~30 RPM. Firing all 13 batch requests simultaneously would send 13 requests in <1 second — exceeding rate limits. Sequential with 1s sleep keeps us at ~4–5 RPM — well under the limit.

Stage 3: Experience Filter (Hard Rules)

L LAYMAN: BLOCKING UNREALISTIC ROLES

Without this filter, the AI was recommending CTO roles to someone with 3 years of experience. Why? Because the AI saw skill overlaps (both CTOs and junior PMs use strategic thinking) and gave it a 65/100.

That's wrong. A CTO role needs 15+ years. No company is hiring a 3-year-experience person as CTO.

So we added explicit rules to the AI's instructions: "If the job requires more than 5 years of experience (3 + 2 buffer), give it a score of 0. If the title says Senior, Lead, Director, VP, CTO, or any other senior level, give it a score of 0."

These rules override everything else. It doesn't matter how good the skill match is — if you can't realistically apply, it's a 0.

T **TECHNICAL: PROMPT HARD CONSTRAINTS VS POST-PROCESSING FILTER**

We chose to implement the experience filter as a prompt constraint (telling the AI "score this 0") rather than a post-processing regex filter (removing jobs with "Senior" in the title after scoring). This is intentional:

- **Prompt constraint:** More robust — catches "Principal PM" (not caught by "Senior" regex), "Head of Product" (not caught by "Director" regex), jobs that say "10+ years required" in the description without any seniority in the title.
- **Regex post-filter:** Would miss nuanced cases; also less transparent — the user wouldn't see these jobs in the report at all to understand why they were excluded.

The `+2` buffer (if experience_years=3, cap at 5) is deliberate — allows the candidate to apply for roles that say "3–5 years required" which are realistic stretches, while excluding "8–12 years required".

The title-based rule catches seniority implied by title without explicit year requirements:

```
"Senior/Lead/Head/Director/VP/CTO/CXO roles that typically
require 8+ years must score 0"
```

Stage 4: Filter, Sort, Rank

T **TECHNICAL: FINAL PROCESSING**

After collecting all scored entries:

```
# Filter: discard below threshold
filtered = [j for j in scored_jobs if j.get("match_score", 0) >= MIN_MATCH_SCORE]

# Sort: best match first
filtered.sort(key=lambda j: j["match_score"], reverse=True)

# Rank: add rank field (1 = best)
for rank, job in enumerate(filtered, start=1):
    job["rank"] = rank
```

`MIN_MATCH_SCORE = 60` is defined in `config.py` — easily tunable. The sort is a Timsort (Python's built-in) — O(n log n), stable, extremely fast for nearly-sorted data.

INPUT

- cv_profile dict

OUTPUT

- List of job dicts, each with added fields:

- all_jobs list (~128 raw job dicts)
- Groq client object

- `match_score` : int 60–100
- `match_reason` : string (AI-written)
- `rank` : int (1 = best)
- Sorted descending by score

128	128	13	22
Raw jobs in	After dedup	AI batch calls	Jobs out (≥60)

6

COMPONENT 6

Excel Generator

The Report Writer

This component takes the dry list of matched jobs and turns it into a beautiful, color-coded, hyperlinked Excel spreadsheet that you can actually use — sorted, formatted, and ready to read in seconds.

Think of it as a graphic designer who takes a plain list and formats it into a professional presentation.
They color the best opportunities green, the decent ones yellow, make the apply links clickable, auto-size every column so nothing is cut off, freeze the header row so it stays visible while you scroll, and add a summary page with statistics.

Excel Generator — The Report Writer

src/excel_generator.py

Why Two Libraries? (pandas + openpyxl)

L LAYMAN: TWO TOOLS FOR ONE JOB

Creating a formatted Excel file is harder than it sounds. You need to: write the data correctly, set cell colors, set font styles, make hyperlinks, auto-size columns, freeze rows. No single Python tool does all of this perfectly.

So we use two tools in sequence:

- **pandas** — a data analysis library that's excellent at taking a list of Python dictionaries and writing them into a perfectly structured Excel file. But it has very limited formatting capabilities.
- **openpyxl** — a library specifically for manipulating Excel files at the cell level. It can color individual cells, set fonts, add hyperlinks, freeze panes. But writing data with it is cumbersome.

We use pandas to write the data, then open that same file with openpyxl and add all the formatting on top. Best of both worlds.

T TECHNICAL: TWO-PASS WRITE STRATEGY

Pass 1 — pandas:

```
df = pd.DataFrame(rows, columns=columns)
df.to_excel(str(filepath), index=False, sheet_name="Jobs")
```

pandas uses its internal xlsxwriter or openpyxl engine (depending on installation) to write a valid .xlsx file. `index=False` prevents pandas from writing a row-number column. The result is a clean, correctly-typed Excel file with headers and data.

Pass 2 — openpyxl:

```
wb = load_workbook(str(filepath))
ws = wb["Jobs"]
# Now apply all formatting cell by cell...
```

`load_workbook()` parses the .xlsx (a ZIP file containing XML) back into memory. We then iterate rows and cells, applying `PatternFill`, `Font`, `Alignment`, and hyperlinks. Finally `wb.save()` re-serializes back to disk.

Color Coding Logic — In Detail

L **LAYMAN: HOW COLORS ARE APPLIED**

Excel cells have a "fill color" property. After reading the match_score from each row, the program decides which color to paint the entire row. Score 80 or above → paint it green. Score 60–79 → paint it yellow. The program literally tells Excel the hex color code for each cell, like giving a painter the exact paint color number.

T **TECHNICAL: CELL FORMATTING IMPLEMENTATION**

```
for row in ws.iter_rows(min_row=2):
    score_cell = row[score_col - 1]
    try:
        score = int(float(str(score_cell.value))) if score_cell.value else 0
    except (ValueError, TypeError):
        score = 0

    if score >= 80:
        fill = PatternFill(fill_type="solid", fgColor="C6EFCE")
        font_color = "276221"
    elif score >= 60:
        fill = PatternFill(fill_type="solid", fgColor="FFEB9C")
        font_color = "9C6500"
    else:
        fill = None
        font_color = "000000"

    for cell in row:
        if fill: cell.fill = fill
        cell.alignment = Alignment(vertical="top", wrap_text=True)
        if cell.column != url_col:
            cell.font = Font(color=font_color)

    # Hyperlink — must override row font color
    url_cell = row[url_col - 1]
    url_cell.hyperlink = str(url_val)
    url_cell.font = Font(color="0563C1", underline="single")
    url_cell.value = "Apply Here"
```

Note the URL column font is set *after* the row loop — this ensures the hyperlink blue overrides whatever row color was applied. The Apply URL column is the only column explicitly excluded from row-level font coloring.

The Summary Sheet

L **LAYMAN: THE SUMMARY PAGE**

The Excel file has two tabs (sheets). The first has all the job listings. The second is a summary dashboard that tells you: how many total jobs matched today, which portals contributed how many, which cities have the most openings, and which 5 companies are hiring the most.

This helps you spot patterns — e.g., "Bengaluru has 80% of the openings" or "Company X is hiring aggressively this week."

📥 **INPUT**

- scored_jobs list (from job_matcher)
- cv_profile dict (for candidate name)
- report_date (date object)
- output_dir: "reports/"

📤 **OUTPUT**

- `reports/YYYY-MM-DD_jobs.xlsx`
- Sheet 1 "Jobs": color-coded, hyperlinked
- Sheet 2 "Summary": stats dashboard
- Returns absolute file path string

⚖️ WHY EXCEL? WHY NOT PDF, CSV, OR A WEB DASHBOARD?

Format	Pros	Cons / Why not primary
Excel (.xlsx) ✓	Universal; color, links, sorting; no internet needed; user can add notes	Requires Excel or Google Sheets to open
CSV	Universal text format	No color, no hyperlinks, no formatting whatsoever
PDF	Good for printing	Not interactive; can't click links natively; can't sort/filter
Web dashboard (Flask/Streamlit)	Pretty, interactive	Requires a running server (not free); adds infrastructure complexity
Google Sheets	Free, collaborative	Requires Google API credentials; adds complexity; not universally preferred

7

COMPONENT 7

GitHub Uploader

The Delivery Person

The simplest component but a crucial one. After the Excel file is generated, it needs to be saved somewhere you can access it. This component commits and pushes the file to your GitHub repository.

Think of it as the delivery person who takes the finished report off the printer and puts it in your mailbox every day.
You don't go to the office to pick it up — it's waiting for you in your GitHub repository when you wake up in the morning.

GitHub Uploader — The Delivery Person

src/github_uploader.py + workflow git steps

L LAYMAN: WHAT IS GIT? WHY "COMMIT AND PUSH"?

Git is a version control system — a tool that tracks changes to files over time. GitHub is a website that hosts git repositories (project folders tracked by git).

When you "commit," you're saving a snapshot of the current state of your files — like taking a photo of your project at a moment in time. When you "push," you're uploading that snapshot to GitHub so it's accessible online.

Every time your agent runs, it creates a new Excel file with today's date (e.g. `2026-03-03_jobs.xlsx`). The uploader then commits this file and pushes it to GitHub. Now you can go to `github.com/sushant17112000/ai-job-agent/tree/main/reports` and see all your past reports, one per day, neatly organized.

T TECHNICAL: GIT CLI VS GITHUB API

Why git CLI (subprocess) and not the PyGitHub API or Octokit?

GitHub Actions runners have git pre-installed and pre-authenticated via the `GITHUB_TOKEN` secret injected as a Bearer token into the git credential store. This means `git push` from the runner works automatically with write permissions to the repo — no OAuth flow, no API token management, no base64-encoding binary files.

Using PyGitHub's `repo.update_file()` API would require: reading the Excel binary, base64-encoding it, making a REST call, handling sha conflicts manually. The git CLI is far simpler and already battle-tested in CI environments.

```
# In github_uploader.py
subprocess.run(["git", "add", file_path], check=True)
subprocess.run(["git", "commit", "-m", commit_message], check=True)
# push is done by the workflow YAML step
```

Rebase strategy for concurrent commits: The GitHub Actions bot commits the Excel file to the repo. If you (or another run) also committed something in the meantime, a naive push would be rejected. The workflow does `git pull --rebase origin main` first — replaying any unpushed local commits on top of the latest remote state.

Conditional execution: The uploader only runs when `os.environ.get("GITHUB_ACTIONS") == "true"` — preventing accidental commits during local development/testing.

INPUT

- Absolute file path to .xlsx
- Commit message string

OUTPUT

- Excel file committed to main branch
- Visible at github.com/.../reports/

- git + GITHUB_TOKEN (from runner env)

- Versioned history of all past reports

⚠️ FAILURE MODES

WHAT CAN FAIL	SEVERITY	HANDLING
Nothing to commit (no new report)	None	git diff --staged --quiet exits 0; workflow continues cleanly
Push rejected (remote ahead)	Medium	git pull --rebase first, then push; handles bot commit conflicts
GITHUB_TOKEN lacks write permissions	Fatal	Workflow yaml must have permissions: contents: write set

8

SYSTEM-WIDE DESIGN

Full End-to-End Data Flow

What every piece of data looks like at each stage — from raw PDF bytes to formatted Excel cells. This is the complete journey of your information through the pipeline.

The Complete Data Transformation Journey

Data starts as a PDF and ends as a color-coded Excel file. Here's every transformation in between.

Stage 1 → 2: PDF to Profile

```
INPUT:  cv/resume.pdf (binary file, ~200KB)
          |
          | pdfplumber.extract_text()
          ▼
STAGE A: Raw text string (~3,000 characters)
"Sushant Bhat | Product Manager
 Experience: 3 years at XYZ Corp as PM
 Skills: Python, SQL, Product Strategy, Agile
 Education: B.Tech NIT Srinagar 2021
 Preferred: Bengaluru, Singapore..."
          |
          | Groq API call (Llama 3.3 70B)
          ▼
STAGE B: cv_profile dict (Python)
{
  "name": "Sushant Bhat",
  "target_roles": ["Product Manager", "Business Analyst"],
  "skills": ["Python", "SQL", "Product Strategy", "Agile"],
  "experience_years": 3,
  "education": "B.Tech, NIT Srinagar",
  "preferred_locations": ["Bengaluru", "Singapore", "Hyderabad"]
}
```

Stage 3: Portal HTML to Job Dicts

```
INPUT:  LinkedIn HTML (rendered by Playwright, ~500KB per page)

<li class="jobs-search__results-list-item">
  <h3 class="base-search-card__title">Product Manager - Growth</h3>
  <h4 class="base-search-card__subtitle">Razorpay</h4>
  <span class="job-search-card__location">Bengaluru, India</span>
  <a class="base-card__full-link" href="https://linkedin.com/jobs/view/3927...">
</li>
          |
          | Playwright selectors + _build_job_dict()
          ▼
STAGE C: List of 128 job dicts
[
  {
    "title":    "Product Manager - Growth",
    "company":  "Razorpay",
    "location": "Bengaluru, India",
    "url":      "https://www.linkedin.com/jobs/view/3927...",
    "description_snippet": "We are looking for a PM with...",
    "date_posted": "2 hours ago",
    "portal":   "LinkedIn"
```

```
  },
  { ... },  # 127 more
]
```

Stage 5: Job List to Scored Shortlist

```
INPUT:  128 job dicts + cv_profile
          |
          | _normalize_url() + set deduplication
          ▼
STAGE D: 128 unique job dicts (no duplicates)
          |
          | Groq API — 13 calls, 10 jobs per call
          | Prompt includes cv_profile + hard rules + rubric
          ▼
STAGE E: Groq returns JSON arrays like:
[
  {"job_index": 0, "match_score": 82, "match_reason": "Strong PM role at fintech..."},
  {"job_index": 1, "match_score": 0,  "match_reason": "Senior Director role..."},  ← filtered by har
  {"job_index": 2, "match_score": 71, "match_reason": "Good fit, location matches..."},
  ...
]
          |
          | _safe_score() coercion + filter ≥ 60 + sort + rank
          ▼
STAGE F: 22 ranked job dicts (each with added fields)
[
  {"rank": 1, "match_score": 92, "match_reason": "...", "title": "PM", ...},
  {"rank": 2, "match_score": 88, "match_reason": "...", ...},
  ...
  {"rank": 22, "match_score": 61, ...}
]
```

Stage 6: Ranked Jobs to Excel

```
INPUT:  22 ranked job dicts
          |
          | pandas → writes raw data to .xlsx
          | openpyxl → applies formatting layer
          ▼
OUTPUT: reports/2026-03-03_jobs.xlsx
  Sheet "Jobs":
    Row 1: [Header] Dark navy background, white bold font, frozen
    Row 2: Rank=1, Score=92, Company=Razorpay → GREEN fill, Apply Here link
    Row 3: Rank=2, Score=88, Company=CRED    → GREEN fill
    ...
    Row 12: Rank=11, Score=71, Company=Meesho → YELLOW fill
    ...

  Sheet "Summary":
    Total Jobs: 22
    By Portal: LinkedIn: 22
```

```
Top Location: Bengaluru (14 jobs)
Top Companies: Razorpay, CRED, Meesho, Swiggy, Zepto
```

Error Handling Philosophy

The system is designed to never crash entirely. Every component fails locally and safely.

L **LAYMAN: HOW A MACHINE HANDLES THINGS GOING WRONG**

Imagine a factory assembly line. If the painting station breaks, do you shut down the entire factory? No — you just skip painting, put out a warning light, and let the car move to the next station unpainted. That's better than stopping everything.

This pipeline uses the same philosophy. If LinkedIn blocks the scraper today — no problem, we note it and continue with whatever data we have. If the AI gives back a garbled score — no problem, we default it to 0 and that job simply doesn't make the shortlist. Only truly unrecoverable problems (your CV file doesn't exist, no API key at all) shut down the pipeline.

The Error Hierarchy

Missing field in JSON

Layer	Failure	Severity	Handling	Pipeline Effect
Startup	GROQ_API_KEY not set	Fatal	`sys.exit(1)` with clear error	Pipeline doesn't start
Startup	`cv/resume.pdf` not found	Fatal	`sys.exit(1)` with file path shown	Pipeline doesn't start
CV Parser	Groq JSON invalid after 2 retries	Fatal	Raises `CVParseError`	Cannot score without profile; stop
CV Parser	None	`setdefault()` fills empty default	Continues; field has empty/zero value	
Scraper (any)	Portal blocked / timeout / 403	Warning	`safe_scrape()` returns []	That portal contributes 0 jobs; others continue
All Scrapers	Every portal returns 0 jobs	Warning	Log warning, `sys.exit(0)`	Clean exit — not a failure, just no data today
Job Matcher	Groq returns bad JSON for a batch	Warning	Retry 3×, then skip batch	Those 10 jobs unscored; rest continue
Job Matcher	match_score is malformed string	None	`_safe_score()` returns 0	Job gets score 0 and is filtered out
Job Matcher	No jobs score ≥ 60	Warning	Log warning, `sys.exit(0)`	Clean exit — no report generated

Excel Generator	Cell value type error during formatting	None	try/except per cell; skip formatting	Data correct; formatting may be partially missing
Git Uploader	Push rejected (remote ahead)	Warning	`git pull --rebase` then push	Resolved; file still committed

> **Key principle:** `sys.exit(0)` (success code) vs `sys.exit(1)` (failure code). GitHub Actions marks a run as "failed" only on exit code 1. Empty results (`sys.exit(0)`) are not failures — they're valid, expected outcomes on days when no relevant jobs are posted.

Complete Technology Decision Log

Every choice in this stack was made deliberately. Here's the full reasoning for each.

Decision	We Use	Runner-Up	Full Reason
Programming Language	**Python 3.11**	Node.js, Go	Richest ecosystem for ML/AI/scraping (pdfplumber, playwright, groq, pandas all have excellent Python SDKs). Node.js has Playwright support but weaker PDF and Excel libraries. Go has no mature equivalents.
Compute Platform	**GitHub Actions**	AWS Lambda, GCP Cloud Run, VPS cron	$0 cost, no infrastructure management, pre-authenticated git access, cron scheduling built in, ephemeral clean environment each run prevents state pollution.
LLM Provider	**Groq (Llama 3.3 70B)**	Anthropic API, Gemini, OpenAI	Only provider with free tier that works from GitHub Actions IPs without credit card, with no India-specific quota restrictions. Tried and failed: Anthropic (separate billing), Gemini (quota=0 in India).
Browser Automation	**Playwright (async)**	Selenium, Puppeteer, Scrapy	Async-native (no threading complexity), modern API with auto-wait strategies, first-class Chromium support, active maintenance. Selenium is synchronous and clunkier. Puppeteer is JavaScript-only. Scrapy can't execute JavaScript.
PDF Extraction	**pdfplumber**	PyPDF2, pypdf, pymupdf, textract	pdfplumber preserves reading order for multi-column layouts — critical for modern 2-column CV designs. PyPDF2 scrambles column-to-column text. pymupdf is excellent but GPL licensed. textract requires system dependencies.
Data Writing	**pandas to_excel()**	openpyxl direct, xlsxwriter, csv	pandas handles type conversion, column alignment, and the DataFrame abstraction cleanly. xlsxwriter can't read-back and modify files. openpyxl alone is verbose for writing data.
Excel Formatting	**openpyxl**	xlsxwriter, xlwt	openpyxl supports both read and write of .xlsx, allows post-write formatting modifications, has full cell-level API for fills/fonts/hyperlinks. xlwt is .xls only (old format). xlsxwriter is write-only (can't reopen).
Async Framework	**asyncio (stdlib)**	threading, multiprocessing, trio	Playwright requires asyncio. asyncio is in Python's standard library — no extra dependency. For I/O-bound tasks (network requests, browser waits) asyncio concurrency is equivalent to thread-based parallelism.
Report Storage	**Git commit to repo**	Google Drive API, S3, email	Already authenticated in the runner via GITHUB_TOKEN. Versioned history of every report. No extra service credentials. Reports accessible via standard GitHub web UI.

Output Format	**Excel .xlsx**	PDF, CSV, HTML, Google Sheets	Excel is universally available, supports colors/hyperlinks/sorting, can be opened offline, and allows the user to add their own notes/columns without breaking anything.
Secret Management	**GitHub Secrets**	.env file, hardcoded, AWS Secrets Manager	GitHub Secrets are encrypted at rest, never appear in logs, injected as env vars only at runtime, and are natively integrated with GitHub Actions — zero additional configuration needed.